Glink

DEPLOYMENT GUIDE

Deploying Glink

This guide takes Glink from this repository to a live, HTTPS website. It is written for whoever runs the deployment — a developer or someone comfortable with a terminal. Each step says what to run and how to check it worked.

1 What you're deploying

2 Before you start

3 Configuration

4 Option A: Docker

5 Option B: A Linux server (Ubuntu, systemd, Caddy)

6 Option C: Hosting platforms

7 HTTPS and the reverse proxy

8 Social sign-in

9 Card payments and email

10 First launch

11 Updating to a new version

12 Backups and restore

13 Monitoring and logs

14 Before real users: launch checklist

15 Troubleshooting

16 Growing past one server

Generated from docs/DEPLOYMENT.md

1. What you're deploying

Glink runs as **one Node.js process** that serves both the website and the API on a single port (default `8787`). It keeps its data in two places on disk:

WHAT	DEFAULT LOCATION	SETTING
Database (SQLite, one file)	`server/data/glink.db`	`DATABASE_PATH`
Uploaded videos	`server/data/uploads/`	`UPLOAD_DIR`

That means the host must provide:

- **Node.js 24 or newer** (the database driver and `.env` loading are built into Node 24).
- **A persistent disk** that survives restarts and redeploys. Without it, every deploy wipes all accounts and videos.
- **A long-running process.** Live updates use a connection that stays open. Serverless hosts (Vercel functions, Netlify functions, AWS Lambda) **won't work**.
- **Exactly one instance.** The database file and live updates live inside that one process. Don't scale to two copies (see section 16).
- **HTTPS** in front of it, via a reverse proxy or your platform.

Sizing. 1 vCPU and **2 GB RAM** is a comfortable start. Video uploads are held in memory while they're saved, so a 500 MB upload needs about that much free RAM. On a 1 GB machine, set `MAX_UPLOAD_MB=200`. Disk: plan around 50–100 MB per uploaded video.

2. Before you start

- [ ] A domain or subdomain, e.g. `app.yourbrand.com`, whose DNS you can edit.
- [ ] Where it will run: a Linux server you control (section 5), Docker (section 4), or a platform with persistent disks (section 6).
- [ ] Optional keys, which you can add later:
  - Google, Apple and/or Facebook sign-in (section 8).
  - A **Stripe** account for card payments and creator payouts, and a **Resend** account for email (section 9).
  - `ANTHROPIC_API_KEY` for the AI pre-flight and product analysis.
- [ ] Read section 14, the launch checklist, before you invite real customers.

3. Configuration

All settings are environment variables. On a server, put them in a `.env` file in the project folder; Glink loads it automatically. Real environment variables take precedence. On a platform, enter them in its settings screen. `.env.example` lists every setting with comments.

Required in production

VARIABLE	EXAMPLE	WHY
`NODE_ENV`	`production`	Turns off development helpers, demo data and test funds
`APP_URL`	`https://app.yourbrand.com`	Public address. Used for sign-in redirects and email links. Session cookies are only marked *Secure* when this starts with `https://`

Usually needed

VARIABLE	DEFAULT	NOTES
`PORT`	`8787`	Port the app listens on, behind your proxy
`DATABASE_PATH`	`server/data/glink.db`	Point at your persistent disk, e.g. `/data/glink.db`
`UPLOAD_DIR`	`server/data/uploads`	Same disk, e.g. `/data/uploads`
`MAX_UPLOAD_MB`	`500`	Keep it in step with the proxy limit (section 7) and your RAM

Optional

VARIABLE	NOTES
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	“Continue with Google”
`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`	“Continue with Facebook”
`APPLE_CLIENT_ID`, `APPLE_TEAM_ID`, `APPLE_KEY_ID`, `APPLE_PRIVATE_KEY`	“Continue with Apple” (HTTPS only)
`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_CURRENCY`	Card top-ups for brands and bank payouts for creators (section 9). Currency defaults to `usd`
`RESEND_API_KEY`, `EMAIL_FROM`	Sends password-reset and team-invite emails (section 9)

VARIABLE	NOTES
`ANTHROPIC_API_KEY`	AI checks of video frames and product pages. Everything works without it
`SEED_DEMO`	`true` creates demo accounts with password `demo1234` on an **empty** database. Default: **off** in production. Use it only on staging
`ALLOW_TEST_FUNDS`	`true` lets brand owners add pretend money, and gives new brands a $5,000 test credit. Default: **off** in production. Use only for testing or invoiced pilots

Never commit `.env`; it is already in `.gitignore`.

---

4. Option A: Docker

The repository includes a `Dockerfile`. It builds the web app, keeps only production dependencies, runs as a non-root user, stores data in `/data`, and has a health check.

> The Dockerfile was written for this release but wasn't built on the development machine (Docker wasn't installed there). The same steps it runs were tested directly: clean install, build, removing dev dependencies, starting in production mode. Build it once on staging before relying on it.

```
docker build -t glink .

docker volume create glink-data
docker run -d --name glink --restart unless-stopped \
  -p 127.0.0.1:8787:8787 \
  -v glink-data:/data \
  --env-file .env \
  glink
```

`NODE_ENV`, `PORT`, `DATABASE_PATH` and `UPLOAD_DIR` are already set inside the image. Your `.env` only needs `APP_URL` and any optional keys. Binding to `127.0.0.1` keeps the app reachable only through your reverse proxy (section 7).

Or with Docker Compose (`compose.yaml`):

```
services:
  glink:
    build: .
    restart: unless-stopped
    env_file: .env
    ports: ["127.0.0.1:8787:8787"]
    volumes: ["glink-data:/data"]
volumes:
  glink-data:
```

Check it: `docker ps` shows the container as *healthy* after about 20 seconds, and `curl http://127.0.0.1:8787/api/health` returns `{"ok":true,…}`.

---

5. Option B: A Linux server (Ubuntu, systemd, Caddy)

Tested shape: Ubuntu 22.04/24.04. Run these as a user with `sudo`.

1. Install Node.js 24

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt-get install -y nodejs
node -v          # v24.x
```

2. Create a service user and get the code

```
sudo useradd --system --create-home --home-dir /opt/glink glink
sudo -u glink git clone <your-repo-url> /opt/glink      # or copy the project folder there
cd /opt/glink
```

3. Install, build, then drop development tools

```
sudo -u glink npm ci
sudo -u glink npm run build        # typechecks everything and builds the website into dist/
sudo -u glink npm prune --omit=dev
```

4. Configure

```
sudo -u glink cp .env.example .env
sudo -u glink nano .env            # set NODE_ENV=production and APP_URL=https://your-domain,
plus any keys
sudo chmod 600 /opt/glink/.env
```

5. Run it as a service. `deploy/glink.service` is ready to use.

```
sudo cp deploy/glink.service /etc/systemd/system/glink.service
sudo systemctl daemon-reload
sudo systemctl enable --now glink
systemctl status glink             # should say "active (running)"
curl http://127.0.0.1:8787/api/health
```

6. Put HTTPS in front. Install Caddy (`sudo apt install caddy`, following caddyserver.com/docs/install). Copy `deploy/Caddyfile` to `/etc/caddy/Caddyfile`, replace `glink.example.com` with your domain, then `sudo systemctl reload caddy`. Caddy obtains the certificate automatically once your domain's DNS points at the server.

7. Firewall. Only web traffic and SSH should be open:

```
sudo ufw allow OpenSSH && sudo ufw allow 80,443/tcp && sudo ufw enable
```

Port 8787 stays closed to the internet. The app trusts the proxy's `X-Forwarded-For` header for rate limiting, so it must only be reachable through the proxy.

---

6. Option C: Hosting platforms

Any platform works if it offers a **persistent disk**, runs **one long-lived instance**, and supports **Node 24 or Docker**. Use these settings:

SETTING	VALUE
Build command	`npm ci && npm run build`
Start command	`npm start`
Health check path	`/api/health`
Instances	exactly 1 (no autoscaling)
Disk	mount it (e.g. at `/data`) and set `DATABASE_PATH=/data/glink.db`, `UPLOAD_DIR=/data/uploads`
Environment	`NODE_ENV=production`, `APP_URL=https://…` and your keys

Platform notes (check each platform's current docs; their screens change):

- **Render:** follow `docs/RENDER.md`, which has a step-by-step guide plus a ready-made `render.yaml` Blueprint. Note that on Render the build command must be `npm ci --include=dev && npm run build`.
- **Railway:** deploy from the repo, add a *Volume* mounted at `/data`, and set the variables. Railway provides HTTPS.
- **Fly.io:** `fly launch` detects the Dockerfile. Create a volume (`fly volumes create glink_data --size 3`) and mount it in `fly.toml` with `[mounts] source = "glink_data"` and `destination = "/data"`. Keep the machine count at 1.

Static or serverless hosting (Vercel, Netlify, GitHub Pages, Cloudflare Pages) can't run Glink on its own.

---

7. HTTPS and the reverse proxy

HTTPS isn't optional. Without it, session cookies aren't marked Secure, Apple sign-in refuses to work, and logins travel unencrypted.

Two ready-made configs are in `deploy/`:

- **`deploy/Caddyfile`**: the simplest option. It gets and renews the certificate itself and allows 500 MB uploads.
- **`deploy/nginx.conf`**: for existing nginx setups. Add a certificate with `sudo certbot --nginx -d your-domain`.

Whichever proxy you use, it must:

1. **Allow large request bodies** (`client_max_body_size 500m` in nginx), or video uploads fail with "413".
2. **Not buffer live updates.** `/api/events` is a long-lived stream. The nginx config turns buffering off for it; Caddy handles this automatically.
3. **Pass `X-Forwarded-For`**, so rate limiting sees real visitor addresses.
4. **Forward everything** (including `/api/*`) to `127.0.0.1:8787`. The app serves the website and the API itself.

Check it: open `https://your-domain`, sign up, and in your browser's developer tools confirm the `glink_session` cookie shows **Secure** and **HttpOnly**.

---

8. Social sign-in

Each provider shows up on the login and sign-up pages once its keys are set. The callback (redirect) URL to register with each provider is:

```
https://YOUR-DOMAIN/api/auth/oauth/google/callback
https://YOUR-DOMAIN/api/auth/oauth/facebook/callback
https://YOUR-DOMAIN/api/auth/oauth/apple/callback
```

Google: Google Cloud Console → *APIs & Services* → *OAuth consent screen* (fill in app name and support email, add scopes `email` and `profile`) → *Credentials* → *Create credentials* → *OAuth client ID* → type *Web application*. Add the Google callback URL above as an authorised redirect URI, then copy the client ID and secret into `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`. Publish the consent screen so people outside your test users can sign in.

Facebook: developers.facebook.com → *Create app* → add *Facebook Login*. Under its settings, add the Facebook callback URL as a *Valid OAuth Redirect URI*, and make sure the `email` permission is requested. Copy the App ID and App Secret into `FACEBOOK_APP_ID` and `FACEBOOK_APP_SECRET`. Switch the app to *Live* mode when you launch.

Apple (requires an Apple Developer account and HTTPS):

1. *Identifiers* → create an **App ID** with *Sign in with Apple* enabled.
2. *Identifiers* → create a **Services ID** (e.g. `com.yourco.glink.web`). Enable *Sign in with Apple*, then configure your domain and the Apple callback URL. This ID is `APPLE_CLIENT_ID`.
3. *Keys* → create a key with *Sign in with Apple*, then download the `.p8` file. Its Key ID is `APPLE_KEY_ID`; your Team ID (top right of the developer portal) is `APPLE_TEAM_ID`.
4. Put the key in `.env` on one line, in double quotes, with `\n` for line breaks: `APPLE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIGT…\n-----END PRIVATE KEY-----"`

Restart Glink after changing keys. `curl https://YOUR-DOMAIN/api/health` lists the enabled providers under `"providers"`.

How accounts are matched: the first social sign-in creates an account. If the provider's *verified* email matches an existing account, the two are linked. Unverified emails are refused.

---

9. Card payments and email

Stripe: brands pay in, creators get paid out

Glink uses **Stripe Checkout** for brand top-ups and **Stripe Connect (Express accounts)** for creator payouts. With the keys set, Billing shows *Add funds* (card payment) and Earnings shows *Set up payouts*. Without them, the app behaves exactly as before.

How the money moves: a brand pays on Stripe's page → Stripe confirms the payment to Glink (the *webhook*) → the wallet is credited, once per payment. When a creator withdraws, Glink sends a Stripe **transfer** from your Stripe balance to their connected account, and Stripe pays it to their bank.

1. **Create the account** at dashboard.stripe.com. Stay in **test mode** (the toggle at the top) until everything works.
2. **Turn on Connect:** *Connect* → *Get started*, choose a **platform/marketplace** that pays out to others with **Express** accounts. Fill in your platform profile and branding; creators see it during setup.
3. **API key:** *Developers* → *API keys* → copy the **Secret key** (`sk_test_…`) into `STRIPE_SECRET_KEY`.
4. **Webhook:** *Developers* → *Webhooks* → *Add endpoint*:
   - URL: `https://YOUR-DOMAIN/api/stripe/webhook`
   - Events: `checkout.session.completed`, `checkout.session.async_payment_succeeded` and `account.updated`

- Also tick **Listen to events on Connected accounts**, so `account.updated` arrives when a creator finishes setup. Some dashboard versions do this as a second endpoint with the same URL; that's fine.
- Copy the **Signing secret** (`whsec_…`) into `STRIPE_WEBHOOK_SECRET`. With two endpoints there are two secrets. Glink checks one, so use a single endpoint where your dashboard allows it.

5. **Currency:** `STRIPE_CURRENCY=usd` by default. Use the currency your Stripe account settles in.
6. Restart Glink. `/api/health` should now show `"payments":"stripe"`.

Testing locally. Install the Stripe CLI and run `stripe listen --forward-to localhost:8787/api/stripe/webhook`. It prints a `whsec_…` secret for your `.env`. Pay with the test card `4242 4242 4242 4242`, any future date and any CVC.

Transfers need an available balance. Card payments take a couple of days to become *available* in Stripe, and transfers to creators come out of that available balance. If it's short, the creator sees a polite "payouts are temporarily delayed" message and keeps their earnings. In test mode, top up with the card `4000 0000 0000 0077`, which makes the funds available immediately.

Creators outside your country. Creator accounts are created in the creator's country. Cross-border payouts depend on your platform's country and Stripe's supported regions; check Stripe's Connect cross-border payouts docs for your setup. If Stripe can't create an account for a country, the creator sees Stripe's reason.

Going live: finish Stripe's account activation, switch the dashboard to live mode, create the live API key and live webhook (same URL and events), and replace both values in your settings.

Resend: password resets and team invites

1. Sign up at resend.com and **add your domain** (*Domains → Add domain*). Add the DNS records it shows and wait until it says *Verified*.
2. *API Keys → Create API key* (sending access) → `RESEND_API_KEY`.
3. `EMAIL_FROM` is the sender, on the verified domain, e.g. `Glink <hello@yourdomain.com>`.
4. Restart Glink. `/api/health` should show `"email":true`. Request a password reset to test it.

Every email is still copied to the outbox table. If Resend refuses one, the error is saved there and printed to the log, and the user's request still succeeds.

---

10. First launch

1. Start Glink. The database and tables are created automatically on first start.
2. `curl https://YOUR-DOMAIN/api/health` should return `{"ok":true,…,"demo":false,"testFunds":false,"payments":"stripe","email":true}`. In production, `demo` and `testFunds` should be `false` unless you turned them on on purpose.
3. Open the site and **sign up** with your own email. Choose *I'm a Brand* and set up your brand; you become its **Owner**.
4. Invite teammates from *Team* (Managers can launch and approve; Reviewers can approve but can't spend). They join automatically when they sign up with the invited email, and get an invitation email once Resend is set up (section 9).
5. Creators sign up from the home page ("Join as a creator").

Staging site? Set `SEED_DEMO=true` on a fresh database to get the demo brand and creators (all with password `demo1234`). Never do this on a production database that real people use.

---

11. Updating to a new version

Database changes are applied automatically at startup: each migration runs once, in order. Updating looks like this.

Linux server

```
cd /opt/glink
sudo -u glink npm run backup                # always back up first (section 12)
sudo -u glink git pull
sudo -u glink npm ci && sudo -u glink npm run build && sudo -u glink npm prune --omit=dev
sudo systemctl restart glink
curl http://127.0.0.1:8787/api/health
```

Docker: `docker build -t glink . && docker stop glink && docker rm glink`, then the same `docker run` as before. The data volume is reused.

Platforms: push to the connected branch; the platform rebuilds and restarts.

Expect a few seconds of downtime during the restart, because there is only one process. Open pages reconnect live updates automatically.

---

12. Backups and restore

What to back up: the database file and the uploads folder.

Database: run `npm run backup` from the project folder. It takes a consistent copy while Glink is running (SQLite `VACUUM INTO`) and keeps the newest 14. Copies go to `server/data/backups/`, or `BACKUP_DIR` if set.

Run it every night with cron (as the `glink` user, `crontab -e`):

```
15 3 * * * cd /opt/glink && npm run backup >> /opt/glink/server/data/backup.log 2>&1
```

For Docker, run it inside the container: `docker exec glink npm run backup`. The backups land on the `/data` volume.

Uploads: copy the uploads folder with your usual tool, e.g. `rsync -a /opt/glink/server/data/uploads/ backup-host:/backups/glink-uploads/`. Or use restic, or sync to object storage.

Keep copies off the server, for example in a nightly upload of `server/data/backups/` to cloud storage. A backup on the same disk doesn't survive a lost disk.

Restore

```
sudo systemctl stop glink
cp /path/to/glink-2026-…db /opt/glink/server/data/glink.db
rm -f /opt/glink/server/data/glink.db-wal /opt/glink/server/data/glink.db-shm
sudo systemctl start glink
```

Practise a restore on staging once, so you know it works before you need it.

---

13. Monitoring and logs

- **Health:** `GET /api/health` returns HTTP 200 when the app is up. Point an uptime monitor (UptimeRobot, Better Stack, your platform's checks) at it.
- **Logs:**
  - **Server:** `journalctl -u glink -f`.
  - **Docker:** `docker logs -f glink`.
  - **Platforms:** the platform's log view.
  - **What you'll see:** startup lines, errors, and any email that would have been sent. The line `ExperimentalWarning: SQLite is an experimental feature` is expected on Node 24 and harmless.
- **Disk space:** videos accumulate in `UPLOAD_DIR`, so alert at 80% full.
- **Background jobs** run inside the app every minute: they release pending earnings after three days and return unused bonus budget. Nothing extra to schedule.

---

14. Before real users: launch checklist

Everything else is production-ready. These items are the gap between "deployed" and "open to customers".

Must do

- [ ] **Email:** Resend keys set and a test password reset received (section 9). Without them, reset links only reach the server log.
- [ ] **Payments:** Stripe **live** keys and live webhook set, Connect activated, and one real top-up plus one creator payout tried end to end (section 9). Without Stripe, brands in production start at $0 and can't add funds.
  - *Closed pilot alternative:* if you invoice pilot brands yourself, set `ALLOW_TEST_FUNDS=true` so owners can credit their wallet, and settle creator payouts manually.
- [ ] `NODE_ENV=production`, an `https://` `APP_URL`, and `SEED_DEMO` not set to `true`.
- [ ] HTTPS working, with the session cookie showing *Secure* (section 7).
- [ ] Nightly backups running, a copy stored off the server, and one test restore done (section 12).
- [ ] Port 8787 not reachable from the internet (section 5, step 7).

Recommended

- [ ] Social sign-in providers switched to live/published mode (section 8).
- [ ] An uptime monitor on `/api/health`.
- [ ] Terms of service and a privacy policy linked from the site. You'll be storing people's videos and emails.
- [ ] Decide who on your team gets the brand **Owner** role.

15. Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
`Cannot find package 'tsx'` on start	Dependencies weren't installed. Run `npm ci` (then `npm run build` and `npm prune --omit=dev`). `tsx` is a production dependency and must stay.
`No such built-in module: node:sqlite`	Node is too old. Install Node 24+.
Site loads but every page is blank / 404	The website wasn't built. Run `npm run build` so `dist/` exists, and start the app from the project folder.
Everything works, then accounts vanish after a deploy	The database isn't on a persistent disk. Set `DATABASE_PATH` / `UPLOAD_DIR` to the mounted disk.
Logged out on every request / can't stay signed in	The site runs on `http://` with an `https://` `APP_URL`, or the reverse way. Make them match and use HTTPS.
"Cross-site request blocked"	The browser's origin doesn't match `APP_URL`. Set `APP_URL` to the exact public address, e.g. `https://app.yourbrand.com` with no trailing slash.
Video upload fails with 413	The proxy's body limit is too low. Raise it (section 7) and keep `MAX_UPLOAD_MB` in step.
Uploads crash the app or the server runs out of memory	Not enough RAM for large uploads. Add RAM or lower `MAX_UPLOAD_MB`.
Live updates don't arrive until refresh	The proxy is buffering `/api/events`. Use the nginx block in `deploy/nginx.conf` (`proxy_buffering off`).
Social button missing	Its keys aren't set, or the app wasn't restarted. Check `/api/health` → `providers`.
Google says `redirect_uri_mismatch`	The callback URL registered with Google must exactly match `APP_URL/api/auth/oauth/google/callback`.
Apple sign-in fails	Apple needs HTTPS, a Services ID matching `APPLE_CLIENT_ID`, and the key in double quotes with `\n` line breaks.

SYMPTOM	LIKELY CAUSE AND FIX
Password reset email never arrives	Check `/api/health` shows `"email":true` and that the `EMAIL_FROM` domain is verified in Resend. Failed sends are logged with Resend's reason. Without Resend, the link is in the server log.
Card payment succeeded but the wallet didn't change	The webhook isn't reaching Glink. Check the endpoint URL, the three events and `STRIPE_WEBHOOK_SECRET`; Stripe's webhook page shows each delivery and its response. Locally, keep `stripe listen` running.
Creator stuck on "Finish setting up payouts"	Stripe still needs details from them. *Continue setup* reopens Stripe's form. Make sure the webhook listens to connected-account events.
"Payouts are temporarily delayed"	Your Stripe *available* balance can't cover the transfer yet. Wait for card payments to settle, or in test mode top up with card `4000 0000 0000 0077`.
"Too many attempts" for everyone	The proxy isn't passing `X-Forwarded-For`, so all visitors look like one address. Fix the proxy headers.

16. Growing past one server

One server comfortably handles an early-stage marketplace. When you need more than one instance or zero-downtime deploys, these pieces change:

TODAY	REPLACE WITH
SQLite file	Postgres
Videos on local disk	Object storage (S3, R2, GCS) with signed URLs
Live updates inside the process	Redis pub/sub (or Postgres `LISTEN/NOTIFY`) between instances
Background sweep inside the process	A single scheduled job / queue worker
Uploads held in memory	Direct-to-storage uploads (presigned URLs)

Until then, keep it at one instance and scale the machine up (more RAM and disk) rather than out.